

Basic Programming Concepts

Machine Language, Assembly Language, and High-Level Languages

BSc Computer Science (First Year) · Detailed notes with diagrams · 2-mark & 14-mark questions

Learning objectives. After studying these notes you should be able to:

- Define machine language and explain why it is difficult for humans to use directly.
- Explain assembly language and the role of the assembler.
- Explain high-level languages and why they are easier to use.
- Distinguish a compiler from an interpreter.
- Trace one instruction through all three levels of language.

1. The three levels of programming language

A CPU can only execute instructions in **machine language** — pure binary. Since writing programs directly in binary is impractical, computer scientists have built increasingly human-friendly languages on top of it, each requiring translation down to the level below.

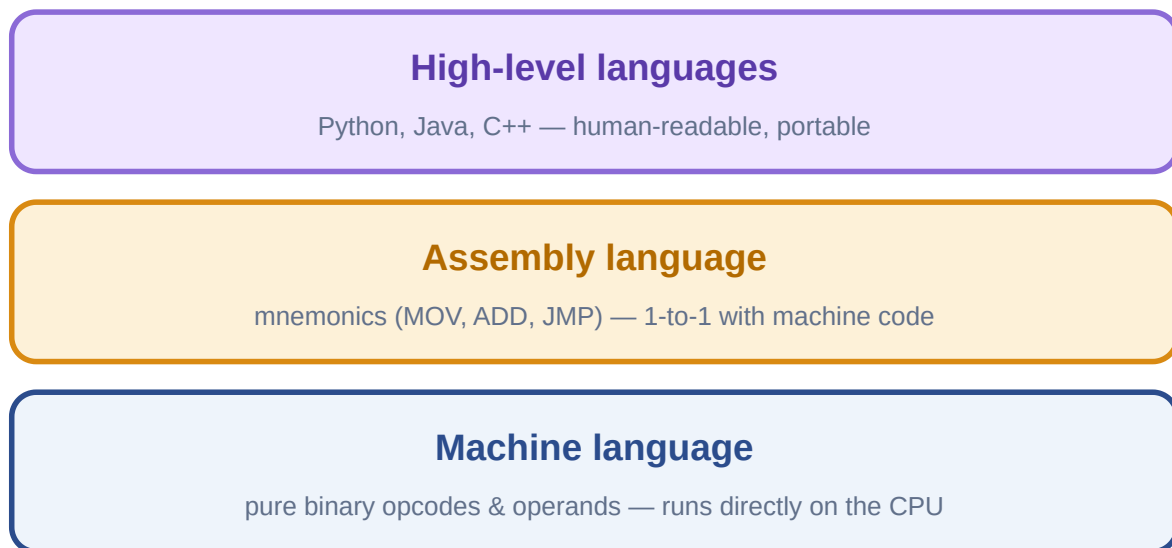


Fig 1. The three levels of programming language, from most human-friendly to what the CPU actually runs.

2. Machine language

Definition. Machine language is a set of binary instructions (patterns of 0s and 1s) that a CPU can execute directly, with no translation required. It is the only language a processor truly 'understands'.

Each machine-language instruction typically has two parts:

- **Opcode** — specifies *which operation* to perform (e.g. load, add, jump).
- **Operand** — specifies *the data or memory address* the operation should act on.

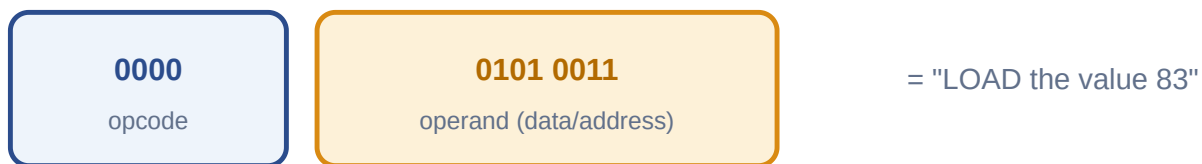


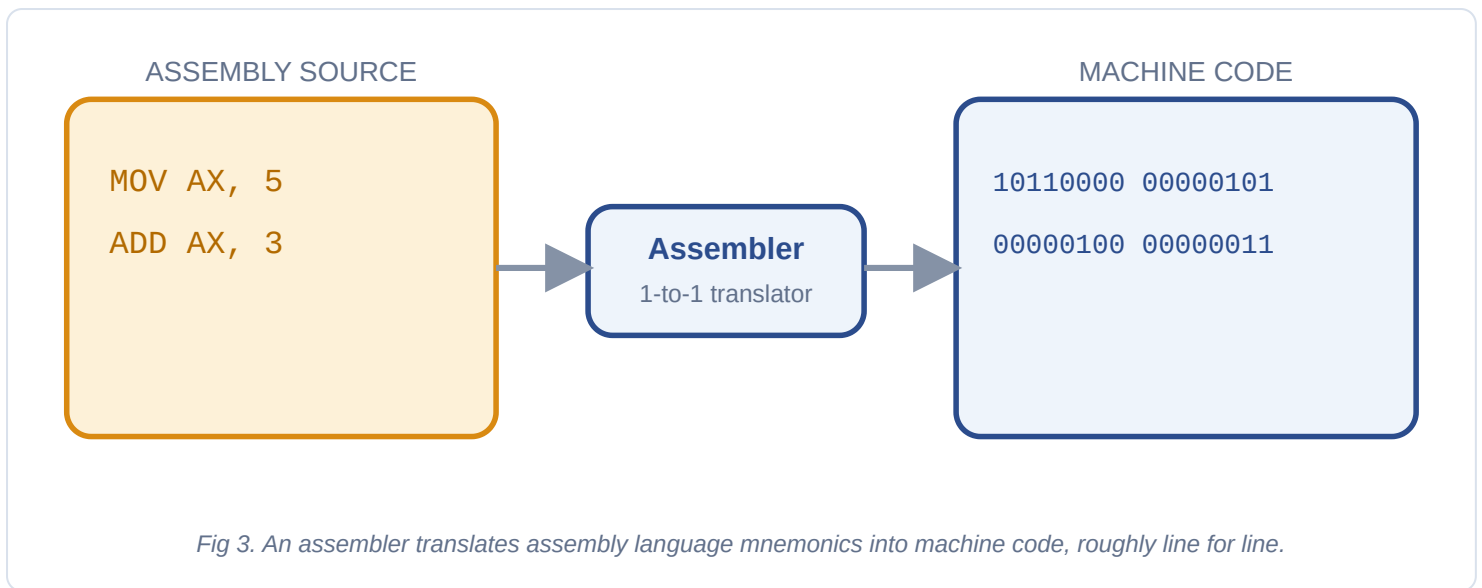
Fig 2. A machine-language instruction is made of an opcode (the operation) and an operand (the data).

Feature	Machine language
Form	Pure binary (0s and 1s)
Translation needed	None — executed directly by hardware
Speed	Fastest possible execution
Readability	Extremely difficult for humans to read, write or debug
Portability	Specific to one CPU's design; not portable

3. Assembly language

Definition. Assembly language replaces raw binary opcodes with short, human-readable **mnemonics** (such as MOV, ADD, JMP), each corresponding almost one-to-one to a specific machine-language instruction.

Because assembly language mirrors machine instructions so closely, the programmer still has to think in terms of the CPU's own registers and memory addresses, and code written for one type of processor generally will not run on another. A special translator program called an **assembler** converts assembly source code into the equivalent machine code.



Feature	Assembly language
Form	Short mnemonics (MOV, ADD, JMP, etc.)
Translator	Assembler (roughly 1-to-1 translation)
Readability	Much better than machine code, still low-level
Portability	Specific to one CPU family

4. High-level languages

Definition. High-level languages (such as Python, Java, C++ and JavaScript) use English-like keywords and structures that are far removed from any specific CPU's design, letting programmers describe *what* they want done without worrying about *how* the hardware does it.

- Far easier and faster for humans to read, write and maintain than assembly or machine code.
- One line of high-level code often expands into **many** machine-language instructions once translated.
- **Portable** — the same source code can often be translated to run on very different types of computer, unlike assembly or machine code.

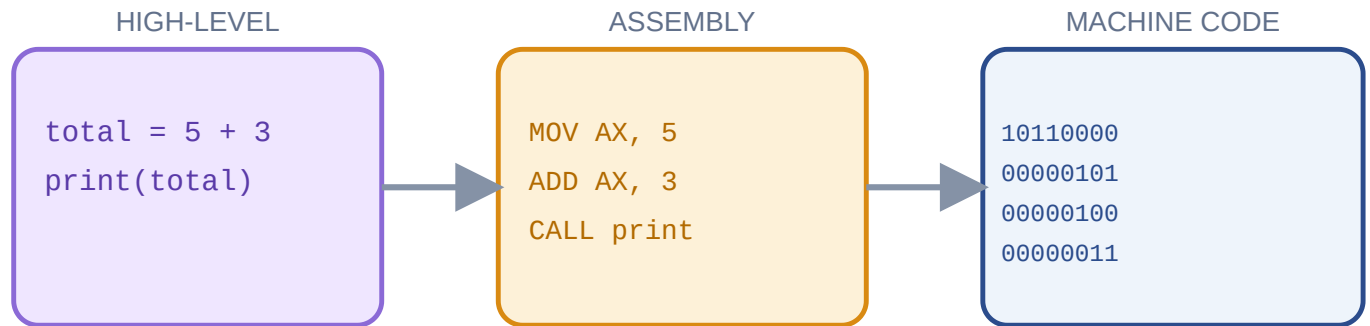


Fig 4. One high-level statement expands into several assembly instructions, and then into many bits of machine code.

5. Compilers and interpreters

A high-level program must still be translated into machine code before (or as) it runs. Two common strategies exist:

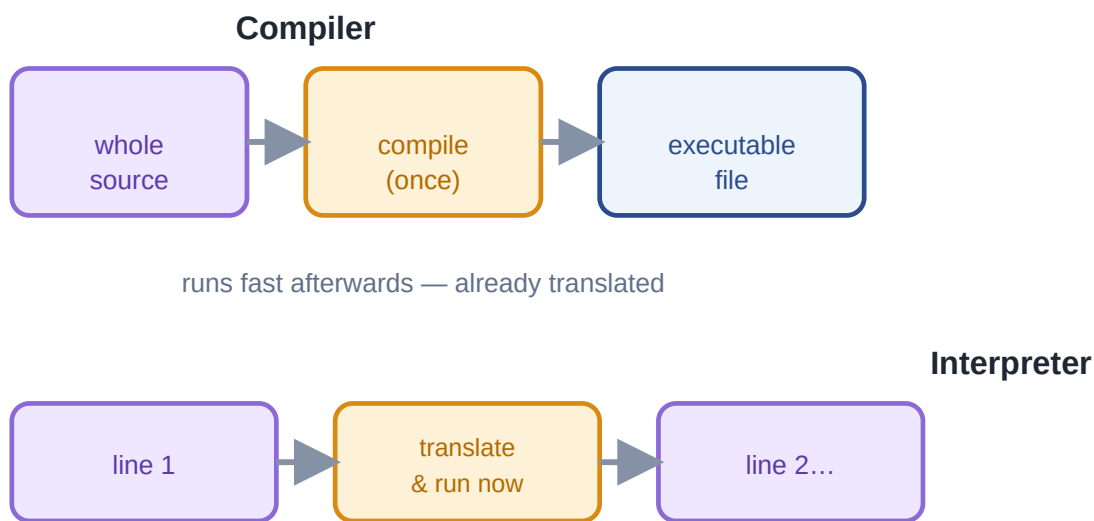


Fig 5. A compiler translates everything up front; an interpreter translates and runs one line at a time.

Compiler. Translates the entire source program into machine code before it is run, producing a standalone **executable file**. All errors are found in one pass, before execution; the resulting program then runs quickly since no further translation is needed.

Interpreter. Translates and executes the program **line by line**, on the fly, without producing a separate executable file. This is convenient for testing and interactive use, but is generally slower, since the same lines are re-translated every time the program runs.

Feature	Compiler	Interpreter
Translates	The whole program, before running	Line by line, while running
Output	A standalone executable file	No separate file — runs directly from source
Execution speed	Fast (already translated)	Slower (translated every run)
Error reporting	All errors found before running	Stops at the first error it reaches
Typical examples	C, C++	Python, JavaScript (typically)

Chapter summary. Machine language is the CPU's native binary tongue; assembly language gives it human-readable mnemonics translated 1-to-1 by an assembler; high-level languages let programmers write in English-like syntax, translated by a compiler (all at once) or an interpreter (line by line). All roads lead back to the same binary instructions the CPU executes.

2-Mark Questions (Short Answer)

Answer in 2–4 lines. These test definitions and single facts.

Q1. Define machine language.**2 marks**

Machine language is a set of binary instructions (0s and 1s) that a CPU can execute directly, without any translation.

Q2. What are the two parts of a typical machine-language instruction?**2 marks**

The opcode, which specifies the operation to perform, and the operand, which specifies the data or memory address the operation acts on.

Q3. Define assembly language.**2 marks**

Assembly language is a low-level language that uses short, human-readable mnemonics (such as MOV, ADD, JMP) that correspond almost one-to-one to machine-language instructions.

Q4. What is an assembler?**2 marks**

An assembler is a translator program that converts assembly language source code into its equivalent machine code.

Q5. Why is assembly language not portable between different types of computer?**2 marks**

Because its mnemonics correspond directly to the instructions of one specific CPU's design, so code written for one processor family generally will not run on another.

Q6. Define a high-level language.**2 marks**

A high-level language uses English-like keywords and structures that are independent of any specific CPU's design, making it easier for humans to read, write and maintain.

Q7. Give two advantages of high-level languages over assembly language.**2 marks**

High-level languages are easier and faster for humans to read and write, and their source code is generally portable across different types of computer, unlike assembly language.

Q8. What is a compiler?**2 marks**

A compiler is a program that translates an entire high-level source program into machine code before it runs, producing a standalone executable file.

Q9. What is an interpreter?**2 marks**

An interpreter is a program that translates and executes a high-level program line by line, without producing a separate executable file.

Q10. Give one advantage of a compiled program over an interpreted one.

2 marks

A compiled program generally runs faster, since it has already been fully translated into machine code before execution begins.

Q11. Give one advantage of an interpreter over a compiler.

2 marks

An interpreter is often more convenient for testing and interactive use, since code can be run and checked immediately, line by line, without a separate compilation step.

Q12. Name one language typically compiled and one typically interpreted.

2 marks

C or C++ is typically compiled. Python is typically interpreted.

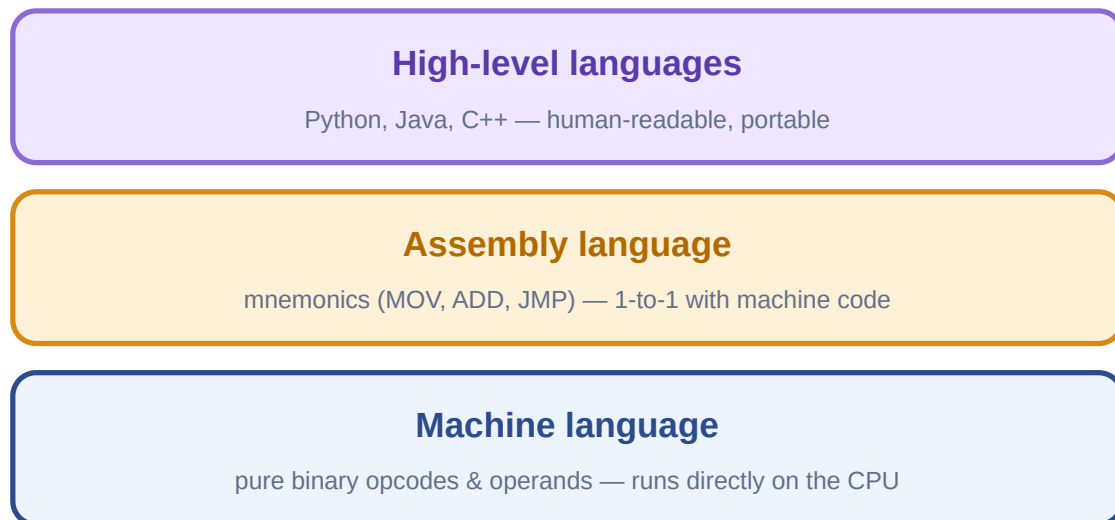
14-Mark Questions (Long Answer)

Long answers should include a definition, well-organised points, at least one labelled diagram, and a short conclusion.

Q1. Explain the three levels of programming languages — machine language, assembly language, and high-level languages — with suitable examples and diagrams.

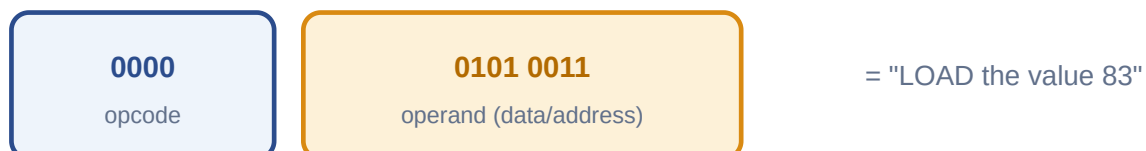
14 marks

Introduction. A CPU can only ever execute instructions written in its own binary machine language. Because writing programs directly in binary is impractical, computer scientists have developed increasingly human-friendly languages, each of which must eventually be translated down to machine code.



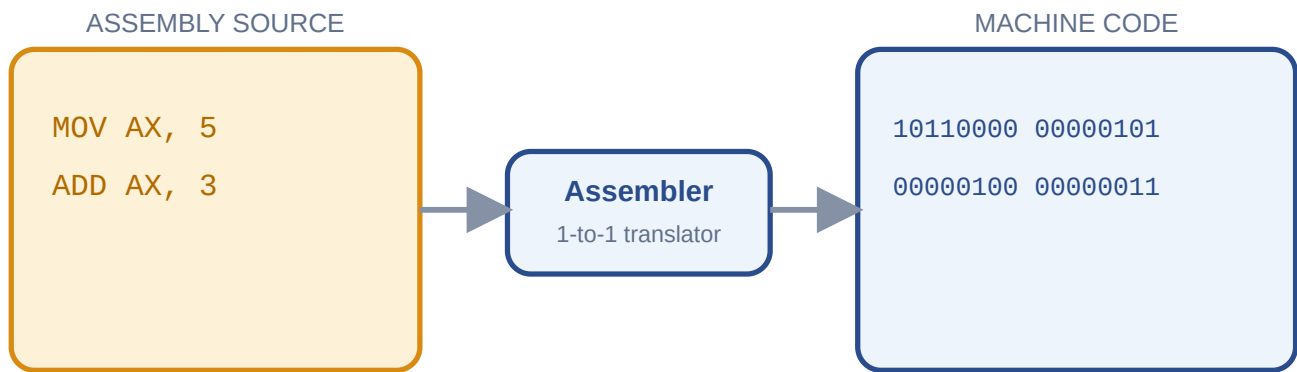
The three levels of programming language.

Machine language. Machine language consists purely of binary patterns (0s and 1s) that a CPU can execute directly, with absolutely no translation needed. Each instruction is made up of an *opcode* (the operation to perform) and an *operand* (the data or address it acts on). While it executes fastest of all, it is extremely difficult for a human to read, write or debug.



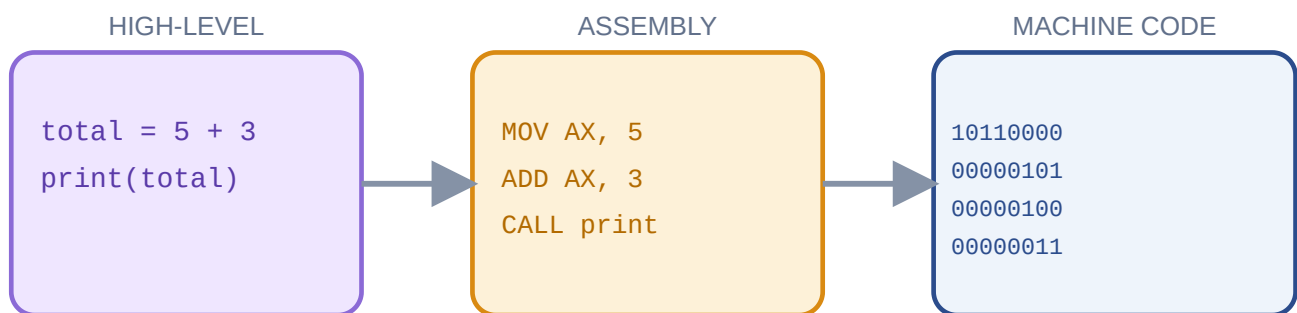
A machine-language instruction: opcode plus operand.

Assembly language. Assembly language replaces these binary opcodes with short, memorable mnemonics such as MOV, ADD and JMP, each standing in almost one-to-one for a machine instruction. It is far more readable than binary but is still tied closely to one specific CPU's design. A translator called an **assembler** converts assembly source into machine code.



An assembler translates assembly mnemonics into machine code.

High-level languages. High-level languages such as Python, Java and C++ use English-like keywords and structures, hiding the details of any particular CPU. A single line of high-level code often expands into many machine instructions once translated, and — unlike assembly or machine code — high-level source code is generally **portable**, meaning it can often be translated to run on very different machines.



One high-level line expands into several assembly instructions, then many bits of machine code.

Comparison table.

Feature	Machine language	Assembly language	High-level language
Readability	Very poor	Poor–moderate	Good
Speed of execution	Fastest	Very fast	Depends on translation method
Portability	None	None (CPU-specific)	High
Translator needed	None	Assembler	Compiler or interpreter

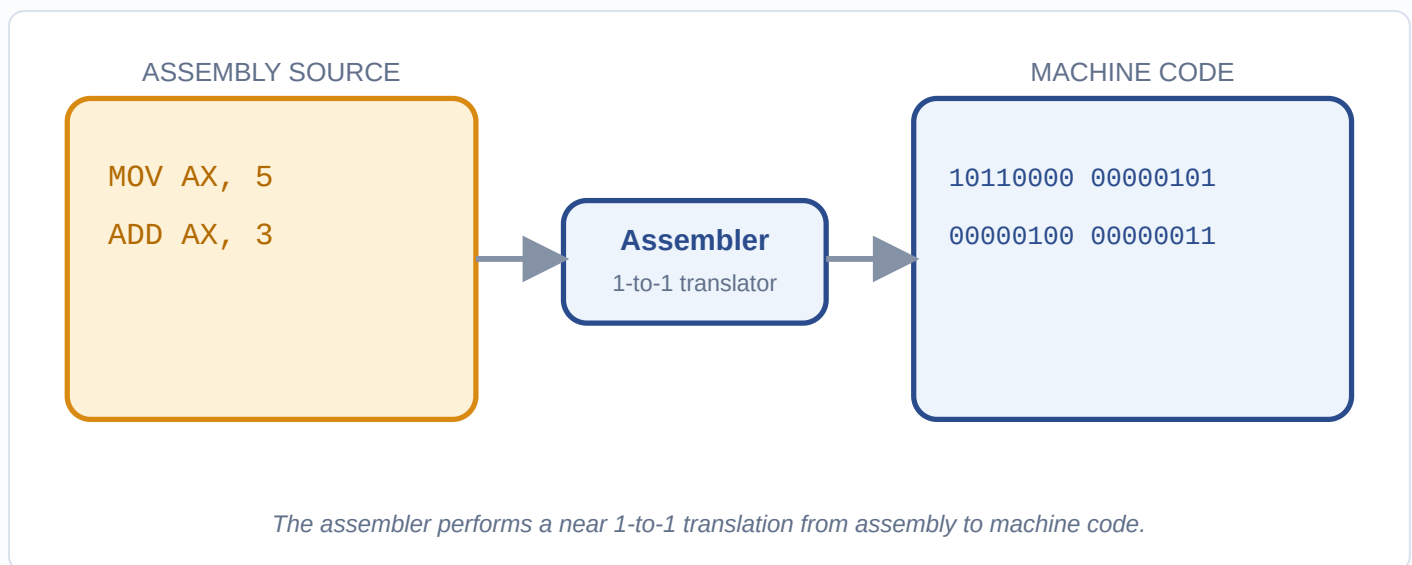
Conclusion. The three levels trade closeness to the machine for ease of human use: machine language is what the CPU runs, assembly gives it readable names, and high-level languages let programmers think in terms of the problem rather than the processor — at the cost of needing an extra translation step.

Q2. What is an assembler? Explain its role, and describe the key features and limitations of assembly language.

14 marks

Definition. An assembler is a translator program that converts assembly language source code into machine code, so that the CPU can execute it. Assembly language itself is a low-level language that represents machine instructions using short, human-readable mnemonics such as MOV, ADD and JMP, instead of raw binary.

How the assembler works. The assembler reads the assembly source code, typically line by line, and produces the exact machine-code equivalent for that specific CPU family. Because each assembly mnemonic corresponds closely to one machine instruction, this translation is close to one-to-one — unlike a compiler, an assembler does not need to make complex decisions about how to translate a line; it mostly performs a direct substitution.



Key features of assembly language.

- Uses short mnemonics instead of binary, making code far more readable than raw machine language.
- Gives the programmer detailed control over the CPU's registers and memory, useful for performance-critical or hardware-specific code.
- Still requires the programmer to think at a low level — close to the actual hardware.

Limitations.

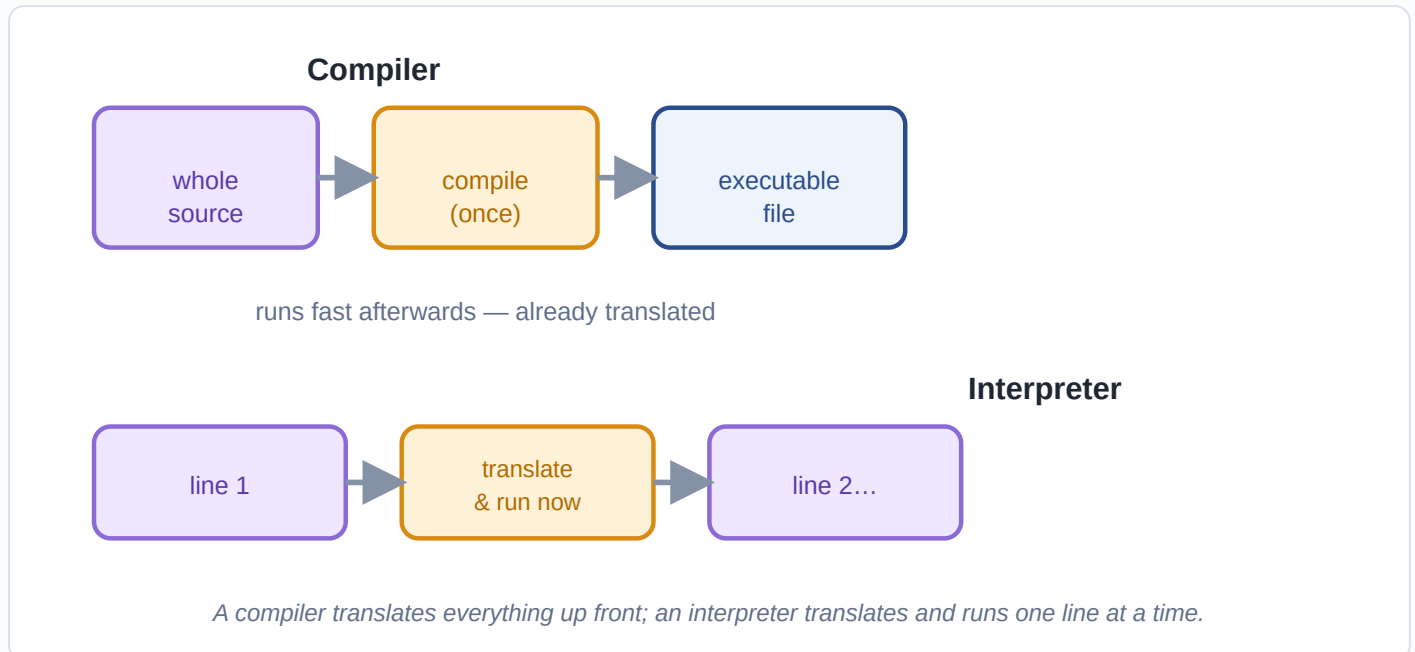
- **Not portable** — assembly code is written for one specific CPU architecture and generally cannot run unchanged on a different one.
- **Time-consuming** — writing and debugging assembly code takes much longer than writing equivalent high-level code.
- **Harder to maintain** — large programs in assembly are difficult to read and modify compared to high-level code.

Conclusion. The assembler's job is to bridge a small but crucial gap — turning readable mnemonics into the exact machine code a CPU needs — but assembly language remains a low-level, CPU-specific tool used mainly where direct hardware control or maximum performance is required.

Q3. Differentiate between a compiler and an interpreter, explaining how each translates a high-level program, and discuss the advantages and disadvantages of each.

14 marks

Introduction. High-level programs must eventually become machine code before the CPU can execute them. Two different strategies achieve this: compiling the whole program in advance, or interpreting it line by line as it runs.



The compiler. A compiler reads the entire source program, checks it thoroughly for syntax and type errors, and translates all of it into machine code in one pass, producing a standalone **executable file**. This executable can then be run directly and repeatedly without needing the compiler or the original source code present. Because translation is done just once, in advance, the resulting program generally runs very fast. Languages such as C and C++ are typically compiled.

The interpreter. An interpreter instead reads the source program one line (or statement) at a time, translating and executing each line immediately before moving to the next. No separate executable file is produced — the program is run directly from its source code every time. This makes interpreted languages convenient for testing, experimentation and interactive use, since a programmer can try a line of code and see its result immediately. However, because translation happens every time the program runs, execution is generally slower than a compiled equivalent. Languages such as Python are typically interpreted.

Advantages and disadvantages.

Feature	Compiler	Interpreter
Error detection	All errors found before running, in one pass	Stops at the first error line it reaches
Execution speed	Fast — already translated	Slower — translated every run
Output	Standalone executable file	No separate file produced
Best for	Finished, performance-critical software	Testing, scripting, interactive development

Conclusion. Neither approach is universally 'better' — a compiler trades a slower build step for fast, repeatable execution, while an interpreter trades execution speed for flexibility and immediate feedback, which is why different languages and use cases favour one over the other.